

图论算法示例

数据结构考试复习 — Dijkstra / BFS / 并查集

2026 年 6 月 18 日

目录

1	Dijkstra 算法（天梯地图）	2
1.1	图的定义	2
1.2	Dijkstra 核心框架（双目标优化）	2
1.3	路径还原	4
1.4	主函数与图的构建	5
2	BFS 广度优先搜索（周游世界）	7
2.1	图的定义	7
2.2	BFS 求最少经停站（无权图最短路径）	7
2.3	BFS 多条件（经停站最少 + 换乘最少）	8
2.4	BFS 求连通分量	10
3	并查集（Union-Find / Disjoint Set Union）	12
3.1	结构体实现（路径压缩 + 按秩合并）	12
3.2	简化数组版（竞赛常用）	13
3.3	应用：统计连通分量	14
3.4	应用：按根节点分组	15
3.5	应用：朋友与敌人关系	15

1 Dijkstra 算法（天梯地图）

1.1 图的定义

```
1 #include <iostream>
2 #include <vector>
3 #include <climits>    // 用 INT_MAX 定义无穷大
4 #include <algorithm> // 用 reverse 还原路径
5 using namespace std;
6
7 // 用一个足够大的数表示无穷大
8 // 使用 INT_MAX/2 而不是 INT_MAX，防止 dist[u] + w 时溢出
9 const int INF = INT_MAX / 2;
10
11 // 边的结构体，表示图中的一条有向边
12 struct Edge {
13     int to;           // 目标节点编号
14     int length;       // 道路长度（距离），单位千米
15     int time;        // 通过该道路所需时间，单位分钟
16 };
17
18 // 节点的结构体，表示图中的一个顶点
19 struct Node {
20     vector<Edge> edges; // 从该节点出发的所有有向边
21 };
```

1.2 Dijkstra 核心框架（双目标优化）

```
1 /**
2  * Dijkstra 求最快到达路线
3  * 主目标：时间最短
4  * 次目标：时间相同时选距离更短的
5  *
6  * @param n      节点总数
7  * @param start  起点编号
8  * @param end    终点编号
9  * @param graph 图的邻接表（vector<Node>）
10 * @return pair<路径向量，总时间>
11 */
12 pair<vector<int>, int> dijkstraTime(int n, int start, int end,
```

```

13                                     const vector<Node>& graph) {
14 // distTime[i]: 起点到节点 i 的最短时间
15 vector<int> distTime(n, INF);
16 // distLen[i]: 在最短时间条件下到节点 i 的最短距离 (打破平局用)
17 vector<int> distLen(n, INF);
18 // parent[i]: 节点 i 的前驱节点, 用于最后还原完整路径
19 vector<int> parent(n, -1);
20 // visited[i]: 标记节点 i 是否已确定最短路径
21 vector<bool> visited(n, false);
22
23 // 起点到自身的距离和时间均为 0
24 distTime[start] = 0;
25 distLen[start] = 0;
26
27 // 主循环: 每次选出一个未访问且时间最小的节点
28 for (int i = 0; i < n; i++) {
29     // 在未访问节点中找 distTime 最小的
30     int u = -1;
31     int minTime = INF;
32     for (int j = 0; j < n; j++) {
33         if (!visited[j] && distTime[j] < minTime) {
34             minTime = distTime[j];
35             u = j;
36         }
37     }
38     // 如果所有剩余节点都不可达, 提前结束
39     if (u == -1) break;
40     visited[u] = true;
41     // 已到达终点, 可以提前退出
42     if (u == end) break;
43
44     // 松弛操作: 用节点 u 更新其所有邻居的距离
45     for (const Edge& e : graph[u].edges) {
46         int v = e.to;
47         if (!visited[v]) {
48             // 情况1: 找到更短的时间 → 直接更新
49             if (distTime[u] + e.time < distTime[v]) {
50                 distTime[v] = distTime[u] + e.time;
51                 distLen[v] = distLen[u] + e.length;
52                 parent[v] = u; // 记录前驱
53             }

```

```

54         // 情况2: 时间相同但距离更短 → 同样更新
55         else if (distTime[u] + e.time == distTime[v]
56                 && distLen[u] + e.length < distLen[v]) {
57             distLen[v] = distLen[u] + e.length;
58             parent[v] = u;
59         }
60     }
61 }
62 }
63
64 // 返回路径和总时间
65 return {getPath(parent, start, end), distTime[end]};
66 }

```

1.3 路径还原

```

1  /**
2   * 从 parent 数组还原完整路径
3   * parent[i] 记录节点 i 的前驱节点，从终点向前追溯到起点
4   */
5  vector<int> getPath(const vector<int>& parent, int start, int end) {
6      vector<int> path;
7      int cur = end;
8      // 从终点开始，通过 parent 向前追溯直到起点
9      // parent[start] == -1 (起点没有前驱)
10     while (cur != -1) {
11         path.push_back(cur);
12         cur = parent[cur];
13     }
14     // 上面得到的是从终点到起点的逆序，反转后得到正确顺序
15     reverse(path.begin(), path.end());
16     return path;
17 }
18
19 /**
20 * 格式化输出路径: start => v1 => v2 => ... => end
21 */
22 void printPath(const vector<int>& path, int start, int end) {
23     cout << start; // 先输出起点
24     // 依次输出路径上的后续节点，用" => "连接
25     for (int i = 1; i < path.size(); i++)

```

```

26         cout << " => " << path[i];
27     }

```

1.4 主函数与图的构建

```

1  int main() {
2      int n, m; // n: 节点数, m: 边数
3      cin >> n >> m;
4
5      // 创建邻接表表示的图, 共 n 个节点
6      vector<Node> graph(n);
7
8      // 读入 m 条边
9      for (int i = 0; i < m; i++) {
10         int v1, v2, oneway, length, time;
11         cin >> v1 >> v2 >> oneway >> length >> time;
12
13         // 正向边 v1 -> v2
14         Edge e1 = {v2, length, time};
15         graph[v1].edges.push_back(e1);
16
17         // oneway == 0 表示双向路, 添加反向边 v2 -> v1
18         if (oneway == 0) {
19             Edge e2 = {v1, length, time};
20             graph[v2].edges.push_back(e2);
21         }
22         // oneway == 1 表示单行道, 不添加反向边
23     }
24
25     int start, end; // 查询的起点和终点
26     cin >> start >> end;
27
28     // 调用 Dijkstra 计算最快路线
29     auto [timePath, totalTime] =
30         dijkstraTime(n, start, end, graph);
31
32     // 输出结果
33     cout << "Time = " << totalTime << ": ";
34     printPath(timePath, start, end);
35     cout << endl;
36

```

```
37     return 0;
38 }
```

2 BFS 广度优先搜索（周游世界）

2.1 图的定义

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>      // BFS 需要队列
4 #include <algorithm> // 用 reverse 还原路径
5 using namespace std;
6
7 // 边的结构体
8 struct Edge {
9     int to;          // 目标站点编号
10    int company;      // 这条边属于哪家公司（用于计算换乘）
11 };
```

2.2 BFS 求最少经停站（无权图最短路径）

```
1 /**
2  * BFS 求无权图中从 start 到各点的最少边数（经停站数）
3  * 核心思想：BFS 按层遍历，第一次访问到某节点时即为最短距离
4  *
5  * @param start 起点
6  * @param n      节点总数
7  * @param graph 邻接表
8  * @return dist 数组，dist[i] = 从 start 到 i 的最少边数，-1 表示不可达
9  */
10 vector<int> bfsShortestPath(int start, int n,
11                             const vector<vector<Edge>>& graph) {
12     // dist[i] = -1 表示尚未访问（也即不可达）
13     vector<int> dist(n, -1);
14     queue<int> q;
15
16     // 起点到自身距离为 0
17     dist[start] = 0;
18     q.push(start);
19
20     // BFS 主循环：队列非空时持续处理
21     while (!q.empty()) {
22         int u = q.front(); // 取队首节点
```

```

23         q.pop();                // 队首出队
24
25         // 遍历节点 u 的所有邻居
26         for (const Edge& e : graph[u]) {
27             int v = e.to;
28             // 如果邻居 v 尚未访问，说明这是到 v 的最短路
29             if (dist[v] == -1) {
30                 dist[v] = dist[u] + 1; // 距离 = 前驱距离 + 1
31                 q.push(v);             // v 入队，后续继续扩展
32             }
33         }
34     }
35     return dist;
36     // 查询结果：dist[end] 即为 start 到 end 的最少经停站数
37     // 若 dist[end] == -1，表示不可达
38 }

```

2.3 BFS 多条件（经停站最少 + 换乘最少）

```

1  /**
2   * PreNode 结构体：记录 BFS 路径上的前驱信息
3   * 用于在找到目标后还原完整路径，以及判断换乘
4   */
5  struct PreNode {
6      int from;           // 前驱节点编号
7      int company;        // 从前驱节点到当前节点经过的公司
8      int transfers;      // 到当前节点为止的累计换乘次数
9  };
10
11 /**
12 * BFS 同时求最少经停站和最少换乘
13 * 优先保证经停站最少，经停站相同时取换乘最少的路径
14 */
15 void bfsWithTransfer(int start, int target,
16                     const vector<vector<Edge>>& graph) {
17     int n = graph.size();
18
19     // dist[i]: 从 start 到 i 的最少经停站数
20     vector<int> dist(n, -1);
21     // transfers[i]: 在经停站最少的前提下，到 i 的最少换乘次数
22     vector<int> transfers(n, 1e9);

```



```

23 // pre[i]: 在最优路径上 i 的前驱信息
24 vector<PreNode> pre(n, {-1, -1, 0});
25
26 queue<int> q;
27 dist[start] = 0;
28 transfers[start] = 0;
29 q.push(start);
30
31 while (!q.empty()) {
32     int u = q.front();
33     q.pop();
34
35     // 遍历 u 的所有邻居
36     for (const Edge& e : graph[u]) {
37         int v = e.to;
38         int comp = e.company;
39
40         // 计算到达 v 的新换乘次数
41         int newTransfers = transfers[u];
42         if (u != start) {
43             // 如果当前边的公司与前一站的公司不同, 说明换乘了一次
44             if (pre[u].company != comp)
45                 newTransfers++;
46         }
47         // 注意: 如果 u 是起点, 第一段线路不算换乘
48
49         // 判断是否更新 v 的最优值:
50         // 条件1: v 尚未被访问过
51         // 条件2: 找到了更少的经停站数
52         // 条件3: 经停站数相同但换乘更少
53         if (dist[v] == -1 ||
54             dist[u] + 1 < dist[v] ||
55             (dist[u] + 1 == dist[v] && newTransfers < transfers[v]))
56             {
57                 dist[v] = dist[u] + 1;
58                 transfers[v] = newTransfers;
59                 pre[v] = {u, comp, transfers[v]};
60                 q.push(v);
61             }
62     }
}

```

```

63     }
64
65     // 输出结果
66     if (dist[target] == -1) {
67         cout << "Sorry, no line is available." << endl;
68     } else {
69         // 最少经停站数（不含起点，dist 统计的是边数）
70         cout << dist[target] << endl;
71
72         // 回溯还原完整路径
73         vector<int> path;
74         int cur = target;
75         while (cur != -1) {
76             path.push_back(cur);
77             cur = pre[cur].from;
78         }
79         reverse(path.begin(), path.end());
80
81         // 输出路径全长（含起点和终点）
82         cout << "路径长度：" << path.size() << endl;
83     }
84 }

```

2.4 BFS 求连通分量

```

1  /**
2   * 从一个起点开始 BFS，标记所有可达节点
3   *
4   * @param start      起点
5   * @param adj        邻接表（无权图）
6   * @param visited    访问标记数组（传引用，会修改）
7   * @param traversal   存储遍历顺序（传引用）
8   */
9  void bfs(int start, const vector<vector<int>>& adj,
10         vector<bool>& visited, vector<int>& traversal) {
11      queue<int> q;
12      q.push(start);
13      visited[start] = true;
14
15      while (!q.empty()) {
16          int u = q.front();

```

```

17         q.pop();
18         traversal.push_back(u); // 记录遍历顺序
19
20         // 遍历所有邻接点
21         for (int v : adj[u]) {
22             if (!visited[v]) {
23                 visited[v] = true;
24                 q.push(v);
25             }
26         }
27     }
28 }
29
30 /**
31  * 对非连通图找所有连通分量
32  * 遍历每个节点，如果未访问就以它为起点做一次 BFS
33  */
34 void findAllComponents(int n, const vector<vector<int>>& adj) {
35     vector<bool> visited(n, false);
36     vector<int> traversal; // 按 BFS 顺序存储所有节点
37     int components = 0; // 连通分量计数
38
39     for (int i = 0; i < n; i++) {
40         if (!visited[i]) {
41             bfs(i, adj, visited, traversal);
42             components++; // 每次 BFS 发现一个新的连通分量
43         }
44     }
45
46     cout << "连通分量数: " << components << endl;
47     // traversal 中存储了 BFS 遍历序列
48 }

```

3 并查集 (Union-Find / Disjoint Set Union)

3.1 结构体实现 (路径压缩 + 按秩合并)

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  const int MAXN = 10000;
7
8  /**
9   * UnionFind 并查集结构体
10  * 支持三种操作:
11  *   find(x)  —— 查找 x 所在集合的根节点
12  *   unite(x, y) —— 合并 x 和 y 所在的集合
13  *   same(x, y) —— 判断 x 和 y 是否属于同一集合
14  */
15  struct UnionFind {
16      // parent[i]: 节点 i 的父节点, 根节点的父节点指向自身
17      vector<int> parent;
18      // rank[i]: 以 i 为根的树的高度 (近似), 用于按秩合并优化
19      vector<int> rank;
20
21      /**
22       * 构造函数: 初始化 n 个独立元素
23       * 每个元素的父节点指向自己, 即每个元素自成一族
24       */
25      UnionFind(int n) {
26          parent.resize(n);
27          rank.resize(n, 0);
28          for (int i = 0; i < n; i++)
29              parent[i] = i;
30      }
31
32      /**
33       * 查找元素 x 所在集合的根节点
34       * 同时进行路径压缩: 将路径上所有节点直接指向根节点
35       * 使得后续查找更快 (近似  $O(1)$ )
36       */
37      int find(int x) {
```

```

38     if (parent[x] != x)
39         parent[x] = find(parent[x]); // 递归路径压缩
40     return parent[x];
41 }
42
43 /**
44  * 合并 x 和 y 所在的集合
45  * 按秩合并：将较矮的树挂在较高的树下，防止树退化成链表
46  */
47 void unite(int x, int y) {
48     int rx = find(x), ry = find(y);
49     if (rx == ry) return; // 已经在同一集合，无需合并
50
51     // 按秩合并：rank 小的挂到 rank 大的下面
52     if (rank[rx] < rank[ry])
53         parent[rx] = ry; // rx 的根指向 ry
54     else if (rank[rx] > rank[ry])
55         parent[ry] = rx; // ry 的根指向 rx
56     else {
57         // rank 相等时，合并后 rank 增加 1
58         parent[ry] = rx;
59         rank[rx]++;
60     }
61 }
62
63 /**
64  * 判断 x 和 y 是否属于同一集合
65  * 根节点相同则在同一集合
66  */
67 bool same(int x, int y) {
68     return find(x) == find(y);
69 }
70 };

```

3.2 简化数组版（竞赛常用）

```

1 // 全局父节点数组，parent[i] 表示节点 i 的父节点
2 int parent[MAXN];
3
4 /**
5  * 初始化：每个元素自成一族

```

```

6  * 即每个元素的父节点指向自身
7  */
8  void init(int n) {
9      for (int i = 0; i < n; i++)
10         parent[i] = i;
11 }
12
13 /**
14  * 查找根节点（路径压缩）
15  * 递归查找的同时压缩路径，使树保持扁平
16  */
17 int find(int x) {
18     if (parent[x] != x)
19         parent[x] = find(parent[x]);
20     return parent[x];
21 }
22
23 /**
24  * 合并两个元素所在的集合
25  * 将 x 的根直接指向 y 的根（或反过来）
26  */
27 void unite(int x, int y) {
28     int rx = find(x), ry = find(y);
29     if (rx != ry)
30         parent[ry] = rx; // 将 ry 的根设为 rx
31 }

```

3.3 应用：统计连通分量

```

1  /**
2  * 统计当前图中的连通分量数
3  * 原理：根节点（parent[i] == i）的个数 == 连通分量数
4  */
5  int countComponents(int n) {
6      int cnt = 0;
7      for (int i = 0; i < n; i++)
8          if (find(i) == i) // 自己是根节点 → 一个分量
9              cnt++;
10     return cnt;
11 }

```

3.4 应用：按根节点分组

```
1 // 将所有元素按所属集合分组
2 // 键：根节点编号，值：该组所有元素编号
3 map<int, vector<int>>> groups;
4 for (int i = 0; i < n; i++) {
5     int root = uf.find(i);    // 找到 i 的根节点
6     groups[root].push_back(i); // 将 i 加入对应组
7 }
8
9 // 遍历每个组，处理组内成员
10 for (auto& entry : groups) {
11     int root = entry.first;    // 组标识（根节点）
12     vector<int>& members = entry.second; // 组内所有成员
13     // 处理该组 ...
14 }
```

3.5 应用：朋友与敌人关系

```
1 /**
2  * 朋友关系 → 合并到同一集合
3  * 敌人关系 → 记录到对方的敌人列表中
4  * 查询时分别检查是否同集合、是否是敌人
5  */
6
7 // enemy[i]：存储所有与 i 是敌人关系的节点
8 vector<vector<int>>> enemy(n + 1);
9 // 并查集初始化
10 vector<int> parent(n + 1);
11 for (int i = 1; i <= n; i++) parent[i] = i;
12
13 // 处理 m 对关系
14 for (int i = 0; i < m; i++) {
15     int a, b, t; // a 和 b 的关系类型为 t
16     cin >> a >> b >> t;
17     if (t == 1) {
18         unite(a, b);    // 朋友 → 合并到同一集合
19     }
20     if (t == -1) {
21         enemy[a].push_back(b); // 敌人 → 互相记录
22         enemy[b].push_back(a);
```

```

23     }
24 }
25
26 // 查询 a 和 b 的关系
27 bool isFriend = same(a, b);      // 同一集合内是朋友
28 // 在 a 的敌人列表中找 b
29 bool isEnemy = find(enemy[a].begin(), enemy[a].end(), b)
30                 != enemy[a].end();
31
32 if (isFriend && !isEnemy)      cout << "No problem" << endl;
33 else if (isFriend && isEnemy) cout << "OK but..." << endl;
34 else if (!isFriend && isEnemy) cout << "No way" << endl;
35 else                          cout << "OK" << endl;

```

——祝考试顺利! ——